

1 L01-AI-Intro

1.1 基本概念介绍

- Multimodality

多模态为模型引入了同时处理不同源，比如文字，音频，图像，视频等，的能力，以让模型获得更加丰富的理解。

- Agentic AI

Agentic AI 能够主动采取行动并在现实世界中发挥作用，不仅仅是响应查询。

- 人工智能

人工智能是能够模拟人类智能某些方面的系统和机器。

- 传统编程

人类专家明确地定义规则，程序一步步遵循这些规则

- 机器学习

机器学习是人工智能的一个子集，它使系统能够通过算法从数据中学习，无需显示编程。人类无需编写规则，通过提供数据和相应标签让机器自动学习模式。

- 深度学习

深度学习是机器学习的一个子集，深度学习使用多层神经网络去建模复杂的数据模式。

- 深度学习 vs 机器学习

1. 机器学习在数据量小的结构化数据训练效果好。深度学习需要大量非结构化的数据，比如图像音频文本。
2. 机器学习使用简单的算法，比如线性回归，决策树，支撑向量机。深度学习使用复杂的卷积神经网络或者循环神经网络架构。
3. 机器学习可以在标准的 CPU 训练，且耗时短。深度学习需要在高性能的 GPU 上训练，训练时间长。

4. 机器学习模型更加透明且易解释。深度学习经常作为黑箱，解释困难。

- 自然语言处理

自然语言处理研究计算机如何理解，生成和处理人类的语言。

1.2 人工智能 ethical problems

- Job displacement and economic impact 工作岗位替代和经济影响
人工智能驱动的自动化可能会取代运输、客户服务等行业的工作人员。
- Bias and fairness
人工智能系统可能会从训练数据中即成或放大偏见。
- Autonomous system and safety
Who is responsible when AI causes harm?
- Misinformation and manipulation
生成式 AI 能够创建出逼真的虚假信息。Ethical problem: 虚假信息传播。

2 L02-ML-workflow

2.1 机器学习类别

机器学习包括有监督机器学习和无监督机器学习。有监督机器学习有回归问题和分类问题，分类问题中包括二分类问题和多分类问题。无监督机器学习包括聚类问题。有监督机器学习中，数据集包括其特征以及每个数据的标签；无监督机器学习数据集仅包含标签。

- supervised learning 监督学习

监督学习通过构建基于一个或者多个输入来预测或估计输出的统计模型。训练集中包含特征和响应/标签，模型需要学习到通过新的数据预测响应的方法。

- classification 分类问题
预测离散类别/标签
- regression 回归问题
预测连续的数值
- unsupervised learning 无监督学习
无监督学习是指无需在带有标签的数据上训练模型就可以从数据上提取含义的统计方法。
训练数据只有样本特征，没有标签。目的是找到数据中的模式、分组、结构或者内在关系。
常见任务：降维，聚类，关联规则学习。
- clustering 聚类
在没有预定义标签的情况下对相似的数据进行分组

2.2 机器学习工作流

2.2.1 确认问题类型

2.2.2 数据收集

- manual data collection 手动数据收集
surveys, interviews, manual labelling
- automated data collection 自动化数据收集
使用 APIs 从线上资源自动获取数据
- sensor or IoT data 传感器或物联网数据
从摄像头 GPS 或者温度传感器等物理设备获取数据
- Database and Log Data 数据库和日志数据
从已有的数据库或者系统日志提取数据
- public or open dataset
公开数据集
- crowdsourcing 众包

2.2.3 Data Preprocessing 数据预处理

Data structuring, handle missing values, normalize or standardize features, and encode categorical variables.

2.2.4 Feature Engineering 特征工程

Select, transform or create new features that can improve model performance.

- 选择最相关的特征
- 将已有的特征通过取对数或者多项式或者 interaction terms 进行转换
- 从领域知识中创建新的特征

2.2.5 模型选择

根据问题和数据特征选择合适的模型

- 分类: logistic regression, Naive Bayes, Nearest Neighbor, Discriminant analysis, decision trees, random forest, support vector machine, neural network
 1. 逻辑回归: 可以拟合一个预测属于一类或者另一类二分响应概率的模型
 2. 判别式分析: 假设不同类别依据高斯分布 (对于不同的类别, 都有自己特别突出的特征) 生成数据, 通过寻找特征的线性组合对数据进行分类
 3. K 近邻算法: 根据数据集中待分类对象最邻近样本的类别来划分其类别
 4. 朴素贝叶斯: 假设某类别中某个特征的存在与其他特征相互独立, 基于数据属于某一类别的概率高低进行分类。
- 回归: Linear regression, Lasso regression, GLM, Gaussian Process, decision tree, random forests, gradient boosting
 1. 线性回归: 用于将连续响应变量描述 (目标变量) 为一个或者多个预测变量 (特征) 的线性函数

2. 非线性回归
 3. 广义线性模型 (generalized linear models): 使用将输入的线性组合拟合到输出的非线性函数上。
- 聚类: K-Means, hierarchical clustering 层次聚类, DBSCAN 密度聚类
 - Data size 数据量
 1. 小的数据集, 选择简单的模型如逻辑回归或者支撑向量机
 2. 大的数据集, 选择复杂模型, 如神经网络
 3. 如果小的数据集选择大的模型, 那么会导致模型对训练集过拟合, 因为小的数据集, 特征较少, 大模型的参数量很多, 足以表示小的数据集中普通样本每一个特征的关系, 也可以表示一些特殊样本负样本的特征的关系, 导致过拟合。一个大的数据集选择简单的模型, 那么模型会欠拟合, 因为模型难以拟合数据特征的关系。
 - Feature type 特征类型
 1. 数值数据: 线性模型或者树模型
 2. 分类数据/混合数据: 决策树, 随机森林, 梯度 boosting 模型
 - 如果数据存在噪声或者缺失数据, 则使用基于树的模型更加鲁棒
 - 如果可解释性 (intepretability) 重要, 使用线性回归、决策树或者逻辑回归
 - 如果预测能力比可解释性重要, 使用 gradient boosting 或者神经网络这些复杂的模型
 - Evaluate Performance and Complexity
 1. 从简单的模型开始训练
 2. 使用验证指标进行对比 (RMSE, accuracy, F1)
 3. 若性能不足, 使用复杂的模型
 - 实验与 tune 调优
 1. 使用 cross-validation 比较模型
 2. hyperparameter tuning 超参数调优 (grid search, random search)

2.2.6 Model Deployment 模型部署

将训练好的模型集成到生产环境中用于实际场景

2.2.7 Monitoring and Maintenance 监控与维护

持续监控模型，在有新数据可用或者条件发生改变时对其进行更新

3 机器学习基础

3.1 常见数据质量问题

3.1.1 Missing data 数据缺失

由于数据输入错误，设备故障或者未收集到的数值，一些特征是不完整的

3.1.2 Inconsistent or unstandardized Data 不一致或者不标准化的数据

格式混杂，单位不一致，大小写不规范

3.1.3 Imbalanced data distribution 不平衡的数据分布

一些模型占比过高，导致训练模型存在偏差

3.1.4 Outliers and noisy data 异常值和噪声数据

干扰模型训练的极端值和错误值

3.1.5 Redundant or Collinear Feature 冗余或者共线特征

重复和高度相关的特征会导致模型模型的不稳定性和过拟合。

3.2 数据预处理

3.2.1 缺失值处理

- 填补或者移除缺失值

- Mean/median/KNN imputation(属于数字内插方法：使用简单的算法快速填补数据空缺)；interpolation 插值方法 (牛顿插值，拉格朗日插值，赫米特插值，三次样条插值)

3.2.2 Normalization 和 standardization

移除尺度差异，加速模型收敛。

1. Normalization 归一化 (一般将数值缩放到 0-1 之间) 归一化常常用于有数值范围的数据，比如像素值 (0-255)，一般用于图像数据处理、神经网络

- Min-Max Scaling 最小最大缩放

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

2. Standardize 标准化 (将数据标准化到均值为 0，标准差为 1) 标准化一般用于位置边界的数据，用于线性模型、SVM、逻辑回归等

- Z-score Normalization

$$x' = \frac{x - \mu}{\sigma} \quad \mu \text{ 是特征的均值；} \sigma \text{ 是特征的标准差}$$

3.2.3 Discretization 离散化

离散化将连续的数值变量通过分组他们的值到一个区间 (bins) 中来将连续的数值转换为离散的变量。

如果模型或算法需要离散的输入或者希望减少连续数据中的噪声或者简化连续数据中的关系。

- Equal-Width Binning 等宽分箱

将数据范围划分为宽度相等的区间, 公式为 $Binwidth = \frac{x_{max} - x_{min}}{k}$, k 为分箱的数量。区间除了最后一个是左闭右闭，前面都是左闭右开。

1. 优点

实现简单

2. 缺点

如果数据分布不均，可能会导致空箱

- Clustering-based Binning 聚类分箱

使用聚类算法自动找到相似度高的箱。此方法是数据驱动的，能适应实际分布。

1. 优点

自适应数据模式

2. 缺点

计算复杂性高

- Equal-frequency Binning 等频分箱

每个箱的样本数量相同，区间宽度随数据分布调整。按照数据排序后，均分样本到每个箱

1. 优点

适配偏态数据 (也就是非正态分布数据，数据有右侧拖尾，如薪资、房价)，对于偏态数据中位数通常比平均值更能代表数据的典型水平，避免极端值导致的分布不均

2. 缺点

每个箱的宽度不一样，边界无意义。

3.2.4 Feature Encoding 特征编码

由于模型无法直接处理文本。特征编码将非数字的分类变量转换为数字形式的过程，以便机器学习算法能够对其进行处理。

- label encoding 标签编码

每一个类别都被分配一个唯一的整数值。

1. 优点：

简单，紧凑

2. 缺点：

整数值之间的关系暗示了顺序关系 ($2 > 1 > 0$)，对 nominal categories(名义类别：是无内在顺序、无大小/等级关系的分类变量)产生误导。

3. 适用对象:

Tree model(决策树, XGboost)

• One-Hot encoding 独热编码

每个类别都表示一个二进制向量, 只有一个元素为 1, 其余元素为 0

适用模型: 线性模型, 神经网络

3.2.5 Data Balancing 数据平衡

在许多问题中, 每个类别的样本数量是不均衡的。如果一个类别占主导地位, 模型可能会忽略少数类别, 从而在准确率上准确率较高, 但对罕见事件招呼率很低。

• oversampling 过采样

1. 通过复制或者人工生成新样本来增加少数类样本的数量

2. SMOTE(Synthetic minority Oversampling Technique): 通过在少数类样本之间进行插值来合成少数类别样本

• Undersampling 欠采样

减少多数类样本的数量, 使其与少数类样本数量匹配。

简单且快速, 但是可能会因丢失多数样本而丢失有价值的信息。

1. 优点

能够保存所有数据集

2. 缺点

合成样本与真实数据模式有偏差, 过度学习少数类细节, 导致模型过拟合, 泛化能力下降

• class-Weight Adjustment 类别权重调整

不改变数据的数量, 提高样本数量少的类别的误分惩罚。

公式: $\omega_i = \frac{N}{k * N_i}$

i 为类别, 每一个 i 分配权重 ω_i 。N 是所有样本数量。N_i 是第 i 类样本的数量。k 是类别数量。

1. 优点

能够保存所有数据集

2. 缺点

需合理调整权重参数否则可能引入新偏差

3.2.6 Feature selection/extraction 特征选择和特征提取

减少维度，提升模型训练效率。移除不相关的冗余的特征。PCA 主成分分析，LASSO 岭回归，information gain 信息增益

- LASSO regression

LASSO 会将不重要的特征系数缩小到 0，从而有效筛选关键特征。

- Information gain

衡量每个特征对预测目标的贡献程度。保留信息增益最高的特征。

- Principal component analysis

将现有特征转换或组合成新的特征。

3.3 机器学习的 3 个基本组成部分

机器学习的目标是使计算机能够从数据中自动学习模式，而不是依赖于明确的编程规则。

3.3.1 Model 模型

模型表示输入 x 和输出 y 之间假设的数学关系。定义了 x 到 y 的映射函数 $f(x; \theta)$, θ 是模型待学习的参数，不同的模型意味着不同的假设空间。

- 对于机器学习任务，我们首先要定义输入空间 x ，和输出空间 y ，不同的机器学习任务之间的主要区别是输出空间 y 的形式。

1. 对于 binary classification(二分类问题): $y = \{0, 1\}$ 或者 $y = \{-1, 1\}$

2. 对于多分类问题 (N 分类问题): $y = \{1, 2, \dots, N\}$

3. 回归问题: $y = \mathbb{R}$

4. 输入空间 x 通常表示样本的特征空间

5. 可以表示为 $p_r(y|x)$ 或者 $y = g(x)$

- Linear Model 线性模型

线性模型假设输入 x 和输出 y 是线性关系的，其假设空间为参数化的线性函数族。

对于特征向量 $x = [x_1, x_2, \dots, x_d]^T$ 和权重向量 $w = [w_1, w_2, \dots, w_d]^T$ ， b 是偏置，那么线性模型预测值可以写为如下公式：

$$\hat{y} = w^T x + b$$

- Nonlinear Models 非线性模型

$$\hat{y} = f(x; \theta)$$

3.3.2 Learning Criterion 学习准则/Objective Function 目标函数/loss Function 损失函数

目标函数定义了模型性能的好坏，衡量预测输出与真实输出之间的差异 (discrepancy)。包括均方误差，交叉熵损失等。

- Binary classification 二元分类

二元分类问题，每个样本可以分为正类和负类。因此预测结果可以分成 4 种 (第一位为模型是否预测正确，第二位为模型预测的结果)：

1. True positive(真阳性)：实际为正例，模型正确预测其为正例的结果。
2. False negative(假阴性)：实际为正例，模型错误预测其为负例的结果。
3. False positive(假阳性)：实际为负例，模型错误预测其为正例的结果。
4. True negative(真阴性)：实际为负例，模型正确预测其为负例的结果。

- confusion matrix 混淆矩阵

1. Accuracy(准确率) 准确率衡量所有样本中, 被正确分类的样本所占的比例。 $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$

但在数据不平衡时, 效果较差。

2. Precision(精确率)

精确率衡量的是预测为阳性的样本中, 实际为阳性的比例。 $Precision =$

$$\frac{TP}{TP+FP}$$

高精度意味着当模型预测为阳性时, 它通常是正确的。精确率侧重于减少假阳性。

3. Recall(召回率)

召回率衡量的是模型正确识别出实际正样本所占的比例。 $Recall =$

$$\frac{TP}{TP+FN}$$

高召回率意味着模型成功捕捉到了大多数的阳性样本, 召回率为了减少假阴性。

4. F1-score(F1 分数)

F1 分数代表精确率和召回率达调和平均值, 提供了一个平衡这两个指标的单一分数。 $F1 - score = 2 * \frac{Precision * Recall}{Precision + Recall}$

- K-Fold Cross-Validation(K 折交叉验证)

1. K 折交叉验证将数据集随机划分为 k 个大小相等的子集; 在每一轮验证中选择一个子集作为测试集, 其余 k-1 个子集作为训练集; 此过程重复 k 次, 确保每个子集都作为测试集使用 1 次; 最后模型性能通过对所有 k 轮的结果取平均值评估。

2. 一般 k=10

- 分类算法中的 Training error(训练误差) and Generalization error(泛化误差)

1. 训练误差: 模型对现有的数据集的拟合程度, 衡量模型在已见过的数据上的表现。
2. 泛化误差: 模型对新的未见过的数据进行正确分类的能力, 衡量模型在训练集之外的泛化能力。

- Underfitting 欠拟合 and Overfitting 过拟合

1. 欠拟合：模型无法捕捉训练集数据的潜在模式。可以通过增加特征数量，选择更相关的特征以及使用更复杂的模型结果等方法来解决。
2. 过拟合：模型在训练数据上过于特化，在新的未见过的样本上表现不佳，导致训练误差持续下降，泛化误差开始上升。

3.3.3 Optimization algorithm 优化算法

优化算法通过调整模型参数来最小化/最大化学习准则。包括：梯度下降，随机梯度下降，动量法，Adam，AdamW

4 Artificial Neural network(ANN) 人工神经网络

4.1 从 Neurons(神经元) 到 Perceptrons(感知机)

感知机是线性二元分类器，常用于监督学习，要求样本是线性可分的。
公式为： $y = f(\sum_{i=0}^n w_i x_i + b)$

x 是输入向量，w 是权重向量（突触），b 是 bias（偏置），f 函数是 activation function 激活函数，Y 是输出信号。

- 输入值
 1. 输入值是每个样本的 features，样本在训练过程中按 batch 提供
 2. 更大规模更多样化的输入特征通常会提升模型性能。
- Weights and Bias
 1. 权重和偏置是可学习的参数。网络通在前向传播和反向传播的训练过程中，这两个参数会被调整以达到期望的值或输出。
 2. 权重用于确定每个特征的重要性
 3. 偏置使模型能够移动激活函数。
- weighted sum 加权和
 1. 将样本的各个特征与各自权重相乘， $z = \sum_{w_j} x_j + b$, w 是权重，x 是输入特征

2. 它是激活函数的输入
 3. 在训练过程中通过反向传播不断更新，来最小化损失。
- 激活函数 activaton function
 1. 使用 step function 阶跃函数：如果加权和结果大于 0，就为 1；如果小于 0，就为 0.
 - 前向传播

输入样本特征值，乘以权重，加权和，输入激活函数，得到输出信号
 - 反向传播

损失函数对参数 w 和 b 进行求偏导以最小或最大化损失，使用偏导结果和学习率更新参数
 - 感知器的局限性
 1. Single-layer structure 单层架构感知器是单层架构，只有一层权重。它完全依赖所提供的输入特征，没有能力发现或学习相关的中间表示。
 2. No ability to handle Non-linearity 没有能力处理非线性问题感知器使用线性激活函数，它无法在学习过程中引入非线性。
 - Multi-Layer Perceptron (MLP 多层感知机)

多层感知机具有多层结构，非线性激活函数和各种损失函数，能够处理复杂的非线性问题和多分类问题。

4.2 Activation function 激活函数

激活函数引入了非线性，使模型能够学习超越线性关系的复杂模式。

- Sigmoid 函数
 1. sigmoid 函数产生 0 到 1 之间的输出，适用于 probabilistic interpretations.
 2. $\sigma(x) = \frac{1}{1+e^{-x}}$
- tanh 函数

1. $\tanh$ 函数产生-1 到 1 之间的输出, 常被使用于中心化数据 (也就是数据均值为 0)。

2. $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

- ReLU(Rectified Linear Unit 整流线性单元)

1. 对负输入输出零, 对正输入输出线性值。

2. $\text{ReLU}(x) = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases}$

3. $\text{ReLU}(x) = \max(0, x)$

- Leaky ReLU

1. $\text{ReLU}(x) = \begin{cases} x, & \text{if } x > 0 \\ 0.1x, & \text{otherwise} \end{cases}$

2. $\text{ReLU}(x) = \max(0.1x, x)$

- ELU

1. $\text{ELU}(x) = \begin{cases} x, & \text{if } x \geq 0 \\ \alpha(e^x - 1), & x < 0 \end{cases}$

- Softmax

1. $\text{Softmax}(x) = \frac{e^x}{\sum_i e^{x_i}}$

- GELU

1. $0.5x(1 + \tanh(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)))$

- Maxout

1. $\max(\omega_1^T x + b_1, \omega_2^T x + b_2)$

- 选择的激活函数与输出范围和目标值类型或该层的作用相关

1. 二元分类: Sigmoid 函数
2. 多分类问题: Softmax 函数, 值域为 0 到 1, 结果和为 1
3. 隐藏层: 整数或者混合实数值, 选择 ReLU、Leaky ReLU, GELU
4. Recurrent neural network: 状态取值 (-1,1), 使用 Tanh

4.3 Loss function 损失函数

损失函数是非负实值函数，用于量化模型预测与真实标签之间的差异。

- 0-1 loss function

$$0-1 \text{ 损失函数衡量预测是正确的还是错误的 } L((y, \hat{y})) = \begin{cases} 0, & \text{if } \hat{y} = y \\ 1, & \hat{y} \neq y \end{cases}$$

y 是真实标签， $\hat{y}$ 是预测标签。

1. 缺点

- Non-differentiable: 损失不连续且导数为 0，无法直接通过基于梯度的方法直接优化。
- 无法直接反应模型对预测结果的置信度的高低。

- Cross-Entropy Loss Function 交叉熵损失函数

交叉熵损失衡量两个概率分布之间的差异，量化了预测概率分布与真实分布的匹配程度。 y 是 (一般通过独热向量编码) 真实标签， $\hat{y}$ 是 (通过 sigmoid 或者 softmax) 预测概率

1. binary cross-entropy 二元交叉熵/Logistic 损失函数

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

(a) y_i $y_i \in \{0, 1\}$

(b) $\hat{y}_i$ $\hat{y}_i \in (0, 1)$

(c) 在 sigmoid 之后使用

(d) Log 损失是 0-1 损失的平滑近似，可以产生概率分布。

2. categorical cross-entropy 分类交叉熵

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

(a) i 是样本的索引， c 是类别

(b) $y_{i,c}$ ，如果 c 是正确的类别，则为 1；否则为 0

(c) $\hat{y}_{i,c}$ ，表示第 i 个样本属于第 c 类的概率

- Kullback-Leibler Divergence(KL 散度)

$$Loss = \sum_i P(x_i) \log\left(\frac{P(x_i)}{Q(x_i)}\right)$$

- Hinge Loss(铰链损失函数)

$$Loss = \max(0, 1 - \hat{y}y)$$

1. 适用于支撑向量机模型
2. 不同类别中存在间隔, 关注误分样本和边界样本

- Mean Square Error 均方误差

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

1. 适用于回归任务
2. 计算目标值与预测值的平方平均值
3. 对异常值敏感

- Mean Absolute Error 平均绝对误差

$$MAE = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

1. 采用目标值与预测值的绝对差值, 相比 MSE 对异常值鲁棒性更强。

- Root Mean Square Error 均方根误差

$$MSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

1. 适用于回归任务

- Huber Loss

$$L_{\delta}(y, f(x)) = \begin{cases} \frac{1}{2}(y - f(x))^2, & |y - f(x)| \leq \delta \\ \delta|y - f(x)| - \frac{1}{2}\delta^2, & |y - f(x)| > \delta \end{cases}$$

1. 融合 MAE 和 MSE 的优势, 小误差时使用二次函数损失, 大误差使用线性函数损失

5 Deep Neural Networks(DNN) 深度神经网络

5.1 深度神经网络

- 训练过程

1. 前向传播：计算模型输出和损失
2. 反向传播：计算计算模型梯度，使用链式法则更新模型参数。

$$(a) \Delta w_{ji} = -\eta \frac{dE}{dw_{ji}} E$$

3. 神经网络有很多参数，因此损失函数是一个由这些参数组成的多变量函数，我们要计算每个参数的偏导，以了解每个参数如何影响损失
 4. 如果想知道每一层的参数如何影响损失，我们需要从最后一层输出层一步一步向前求每个参数的偏导数 (占用大量计算资源)
- 参数更新：使用梯度和学习率自适应权重
 - 循环以上过程
 - universal function approximation Theorem 通用函数逼近定理

给定任意连续函数 $f(x)$ ，如果一个 2 层的神经网络有足够的隐藏单元，那么存在一种权重选择，使其能够以任何期望的精度紧密逼近任何连续函数 $f(x)$

1. 大量神经元会导致过拟合风险，需要一些早停或者正则化策略
 2. Automatic differentiation 自动微分
- Scaling laws 缩放定律
 1. Model size: 更大的模型/更多的模型参数通常在任务上表现更好
 2. Data Volume: 使用更多的数据训练，通常会提高准确性
 3. Computation 计算能力：增加计算资源可以训练更深更大的网络
 - 深度神经网络

$$z_i^k = g(\sum_j W^{(k-1,k)}_{i,j} z_j^{(k-1)} + b_i^{(k)})$$

1. 第 k 层的第 i 个神经元等于连接第 k-1 层第 j 个神经元与第 k 层第 i 个神经元的权重乘以第 k-1 层的神经元输出之和加上偏置的数值，最后输入激活函数 g

5.2 Deep Learning 深度学习

5.2.1 Data preparation 数据准备

‘数据被分为训练集，测试集和验证集。训练集用于模型训练。验证集一般用于超参数选择，测试集用于模型泛化能力测试。

- Data Collection & Quality
 1. 确保你有足够多与你要解决问题相关的较高质量的数据
- Data preprocessing
 1. 归一化/标准化：对数据进行标准化，确保模型更快更稳定地收敛
 2. Data Augmentation 数据增强：对于图像任务，可以通过 transformation(比如：旋转，翻转，缩放) 创建更多样的训练数据
- Handling Imbalanced Datasets
 1. 考虑对少数类进行过采样，多数类进行欠采样，
 2. 使用带权重的交叉熵损失，focal loss 的相关的损失函数

5.2.2 Model Design

- Architecture 架构
 1. 基于问题选择什么类型的模型 (CNN 适合图像数据训练；RNN 和 Transformer 适合时序数据)
 2. 选择层数以及层类型 (convolutional 卷积层, fully connection 全连接层, recurrent 循环层)
 3. 越深的模型可以捕捉到越复杂的模式，但是很难训练
- Layer Sizes 层大小

层大小表示 the number of neurons in each layer. 越多的神经元会提高模型能力，但对与小数据集会导致过拟合的风险。

 1. Funnel Rule 漏斗规则

输入->large hidden size->smaller hidden size->output
 2. 数据大小 VS 模型大小

- 参数数量 $< 10 \times$ 样本数量
 - 使用 regularization(dropout 或者 L2) 正则化
 - 从简单的网络开始, 向大网络扩展
- 激活函数
影响模型学习非线性关系的方式。
- 输出层和损失函数
 1. 根据任务选择输出层 size。分类任务: softmax; 回归: 线形层
 2. 损失函数: 损失函数取决于具体任务。分类问题: 交叉熵损失; 回归: 使用均方误差
- 机器学习中的 Convergence
收敛指训练过程中模型参数和损失函数值趋于稳定状态的过程。
 1. 损失值随着迭代不断减小, 参数更新变小。也就是模型学到了足够多的知识。
- 机器学习中的 divergence 发散指训练过程没有朝着稳定改进的方向发展, 而是训练过程变得不稳定
 1. 训练损失没有下降反而上升了
 2. 损失大幅度波动
 3. 参数更新变化很大, 且不稳定 (overshooting 超调)

5.2.3 Training Process 训练过程

- Optimizer 优化器

优化器通过调整模型的参数来最小化损失函数。比如 SGD 随机梯度下降, RMSprop, Adam 等。

1. 训练的目标是通过优化器更新模型参数, 最小化损失函数
$$L.\theta \leftarrow \theta - \eta \frac{dL}{d\theta}$$
2. η 是 learning rate 学习率
3. $\frac{dL}{d\theta}$ 是损失函数对于参数的偏导数

4. 优化器类型

* Stochastic gradient descent(SGD 随机梯度下降)

(a) 根据损失函数对参数的梯度更新权重

$$(b) w_{new} = w_{old} - \eta \nabla L(w)$$

(c) η 是学习率; $\nabla L(w)$ 是梯度

* Momentum 动量法

(a) 使用上一步的梯度平滑权重更新并且可以预防收敛到局部最小值。

(b) 为什么叫动量法, 我们使用之前的速度的动量来冲过当下的局部极小值

$$(c) v_t = \beta v_{t-1} + (1 - \beta) \nabla L(w), w_{new} = w_{old} - \eta v_t$$

(d) 如果 β 小, 那么当前的偏导对当前的梯度影响大; 若 β 大, 那么上一步梯度对当前的梯度影响大

* RMSprop(root mean square propagation 均方根传播)

$$(a) v_t = \beta v_{t-1} + (1 - \beta)(\nabla L(w))^2;$$

$$(b) w_{new} = w_{old} - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla L(w)$$

(c) 将学习率除以梯度的移动平均值。通过除以每一个参数的梯度, 控制每一个参数的步长/学习率, 如果一个参数上一步的更新梯度很大的话, 容易导致更新不稳定, 那么学习率除以 v_t 会导致学习率变小, 避免更新补偿太大导致震荡。

(d) 若梯度小, 参数更新较慢, 那么学习率除以梯度导致学习率变大, 加速收敛。

(e) 在计算梯度是加平方: 消除方向的影响, 关注参数更新的波动程度; 在求梯度时减少计算量。

(f) 可以处理梯度消失的问题

* Adaptive Moment Estimation(Adam, 自适应动量估计)

$$(a) m_t = \beta m_{t-1} + (1 - \beta) \nabla L(w)$$

$$(b) v_t = \beta v_{t-1} + (1 - \beta)(\nabla L(w))^2$$

$$(c) \text{Weight Update: } w_{new} = w_{old} - \eta \frac{m_t}{\sqrt{v_t + \epsilon}}$$

(d) 有 RMSprop 的优点和 Momentum 的优点

– Learning rate 学习率

学习率控制优化器在梯度方向上的步长大小

1. 太高的学习率可能会阻碍收敛，过低的学习率导致训练缓慢
2. 可以使用学习率调度 (步进衰减，指数衰减，余弦退火) 或者自适应学习率

– Batch size 批大小

一个 batch size 表示每一次梯度更新训练的样本数量

1. 较小的 batch size 可以加快泛化速度，更好地适配新数据。因为小批量样本随机性强，导致每次更新参数时可能学习到更多新知识，帮助模型跳出局部最优，避免过度依赖训练数据细节。
2. 大 batch size 加快计算速度，且梯度更稳定。大批量中，GPU 一次性处理多个样本，充分利用其并行计算能力；大量样本梯度平均后波动更小，不会频繁震荡。但是大样本会覆盖大多数的训练集，导致过拟合。

– Number of Epochs 训练轮次

一个训练轮次是使用训练数据集中所有样本进行一次训练的完整训练周期，可以包含多次 batch size。

1. 如果 epoch 数量少，模型可能不能学习到数据中的潜在模式，导致欠拟合
2. 若 epoch 过大，模型可能会深究训练集的细节，导致过拟合，在未见过的新的数据集上泛化能力不佳。
3. 理想的 epoch 次数通过记录模型在每个 epoch 训练完之后在验证集上的性能来确定

– Regularization Techniques 正则化技术

正则化是用于减少神经网络过拟合的技术，dropout, L1 正则, L2 正则，批归一化。

1. Dropout: 在训练过程中，每次前向传播时每个神经元按照一定概率 (20%-50%) 随机丢弃。减少有效神经元数量，同时反向传播时这些神经元值的梯度值也为 0，不影响参数更新，可以防止过拟合。

2. L2 正则 (Weight Decay 权重衰减): 会在损失函数中添加一个惩罚项, 抑制较大的权重值。 $L_{total} = L_{original} + \lambda \sum_i w_i^2$ 。 λ 是正则化因子。减小模型权重, 让模型更平滑, 在加权和, 权重减小, 即使 x 有变化, 输出 y 的波动也比较小。

– evaluation metrics 评估指标

选择合适的评估指标, accuracy, precision, recall, F1 score, AUC-ROC.

1. ROC Curve(Receiver Operating Characteristic Curve)

(a) Y 轴: 真阳性率 True Positive Rate=TPR=Recall= $\frac{TP}{TP+FN}$

(b) X 轴: 假阳性率 False Positive Rate=FPR= $\frac{FP}{TP+TN}$

(c) ROC 曲线上的每一个点对应不同分类阈值的 (假阳性率, 真阳性率)。对于每一个阈值, 求其真阳性率和假阳性率, 若曲线在对角线之上, 则模型效果可以。

2. AUC(Area Under the Curve)

AUC 量化了模型区分正类和负类的整体能力, 曲线下面积越接近 1, 模型效果越好, 越接近 0.5 模型效果越差

6 Convolutional neural networks(CNN 卷积神经网络)

6.1 神经网络特征学习

6.1.1 Hierarchical feature learning 分层特征学习

1. 每一层都学习不同层次的特征

- Shallow layers 浅层 (靠近输入) 学习 low-level patterns 低级模式。(边缘, 角点, 颜色梯度)
- Middle layers 将简单的模式组合成更有意义的结构。(纹理和形状)
- deep layer 学习更高级的抽象特征。(物体部件)

2. 反向传播使网络能自动学习特征

- 网络做出预测
- 将预测结果与真实标签进行比较，得到损失
- 损失被反向传播，告诉每一层调整权重，使输出更接近真实情况
- 每一层都会更新其参数，以检测能够减少损失的模式

3. 传统全连接层在处理图像时面临的两个问题

- (a) 参数过多：对于 $100 \times 100 \times 3$ 的图像，第一个隐藏层的每一个神经元都需要 30000 个独立连接。导致数据量大，且容易过拟合
- (b) lack of local invariance 缺乏局部不变性：图像对象具有局部特征，这些特征在缩放旋转移位等操作下基本不变。若不进行大量数据增强，难以捕捉到这种不变性。

6.2 CNN

卷积神经网络是深度前馈神经网络，具有 local connections 局部连接，weight sharing 权值共享，spatial invariance 空间不变性。

- 深度学习中卷积运算
 1. 深度学习中卷积运算 (torch 库) 直接进行滑动点积，不进行翻转 (numpy 库)。因为深度学习中参数是可学习的，不是固定的滤波器算子；省去翻转操作可以减少一次不必要的计算步骤。
- Padding 填充
 1. border effect 边界效应：图像的中心像素会参与大量的卷积运算，图像的边缘位置参与卷积运算次数较少。比如中心位置的像素能被 5×5 的卷积在不同的卷积元素位置上覆盖，但是边缘像素只会覆盖一次。
 2. 造成图像信息损失：边缘像素的特征无法充分参与卷积计算，导致输出特征图中边缘信息丢失
 3. 图像尺寸变小：卷积后的特征图尺寸可能比输入小
 4. 0-padding：在输入图像的外围补 0，减轻边界效应，同时增加生成的特征图的空间维度

- Pooling Layer 池化层

卷积层之后是池化层。

1. 池化目标

- Spatial dimension reduction(减小空间维度)

池化从过滤区域提取一个值来表示这个区域，因此会减小矩阵的大小，并降低网络计算复杂度。

- Feature generalization(特征泛化)

输入的微小变化和偏移将不会影响池化后的特征图，这是的网络对微小变化具有不变性。

- Increased receptive field(增大感受野)

感受野是 CNN 中某一层神经元能看到的原始输入图像的区域，若卷积核为 3×3 ，那么感受野为图像中的 3×3 区域。再使用 2×2 的池化层，就会变成 6×6 的感受野。

通过对特定邻域像素窗口进行池化，网络能从更大区域捕捉模式，增大感受野。

2. MaxPooling 最大池化

最大池化选择每个池化窗口的最大值并且保留窗口中的主要特征。

- 帮助卷积神经网络关注最重要的模式

- 让模型对噪声具有鲁棒性

3. AveragePooling 平均池化

平均池化选择每个池化窗口的平均值并且保持整体信息的平滑。

- 保留更多的背景信息

- 当提取图像中像素强度分布时有用

4. 池化缺点

(a) 池化会损失一些信息，尤其是一些较弱但是重要的特征（边缘/角点）

(b) 池化是一种不可学习的操作。

- stride 步长 (strided convolution)

步长 $S > 1$ 会跳过输入中的某些位置，降低输出特征图的空间分辨率(下采样)。

优点：

1. 由于卷积是可学习的，虽然步长仍然是下采样，但是学习到了如何以最优方式进行下采样

6.3 训练 CNN

- 构建 CNN 网络
 1. 输入，卷积，激活函数，最大池化，卷积，激活函数，最大池化，全连接层(线性层，激活函数，线性层)，全连接层，输出

7 natural language processing(NLP 自然语言处理) 介绍

7.1 NLP 概述

- NLP 的定义：理解和生成人类语言的计算方法
- 挑战：将语言的 semantics(语义) 和 pragmatics(语用) 融入计算机理解非常困难。
 1. Ambiguity(语言歧义，相同语言不同语义)
 2. Paraphrasing(释义相同语义的不同表达方式)
 3. Socio-Linguistic Variation(社会语言变体：同一语言因使用场景、人群不通，在表达上存在差异)
 4. Multi-Language and Code-Switching(多语言混合和缩写)

7.2 NLP Upstream tasks 上游任务

上游任务是指为下游应用任务打基础的核心预处理任务，他们不直接解决实际业务问题，而是通过处理原始文本、提取基础语言特征，为后续复杂应用提供支撑。

7.2.1 Preprocessing Tasks:Tokenization 分词 & Sentence splitting 分句

- Tokenization 分词: 将文本拆分为称为 token 的小单元。
- N-grams: 由 n 个连续单词组成的序列, 用于捕捉局部语境并预测语言模式
- 句子拆分: 将文本分割成句子
 1. 拆分的句子在句法上是完整的, 并且代表完整的思想
 2. 拆分的句子可以支持需要小文本单元的任务, 比如机器翻译
 3. 拆分的句子支持并行处理的任务。句子分割后, 多个句子是相互独立的, 可同时分配给多个计算核心进行处理。

7.2.2 Language Modeling(LM 语言建模)

- 定义: 基于前文已有的单词, 预测序列中的下一个单词, 以捕捉语言的架构和模式, 理解语境。
- 一个句子的概率, 可以被定义为给定先前符号每一个符号概率的乘积。
- 语言模型的类型:
 1. Statistical Models 统计模型: 概率预测
 2. Neural Models 神经模型: 深度学习模型 (RNN, Transformer)

7.2.3 Part of Speech(POS) Tagging 词性标注

- 目标: 由于词语存在歧义, 因此要确定每个词的 syntactic categories(语法类别:noun,verb...), 以及 additional properties(附加属性: tense 时态)
- Annotated Corpus 标注语料库: Pre-labeled data for training(用于训练的预标注数据)
- 词句的特征:
 1. 基本特征: suffix/prefix 前后缀, sentence position 句子位置, capitalization 大写, punctuation 标点来确定当前词的词性

2. context window(上下文窗口): 融合相邻 token 的特征来决定当前词的词性
3. Previous POS Tags(先前词性标签): 将先前词语的词性预测结果当作特征辅助当前单词的词性判断

- 机器学习预测 POS Tagging

1. 算法: 逻辑回归, 决策树, 随机森林, 支撑向量机
将当前 token, 前一个 token, 后一个 token 的特征映射到词性标签, 每个词元单独分类。
 - (a) 先对原始文本分句, 再进行分词。标签编码: 将词性标签变成整数索引, 便于模型计算损失, 构建成标签映射字典, 训练后用于反向解码。模型输入单 token 特征向量 (基础特征 + 语境窗口特征: 当前词的特征向量 = 自身词形特征 + 前词的词形特征 + 后词词形特征)。经过模型, 预测当前模型的词性标签概率分布, 求损失。
 - (b) 模型通过学习标注好的数据集中数据规律 (比如: I 会标注成 PRP 表示人称代词, like 标注成 VBP 表示动词原形等), 建立特征组合与词性标签的对应关系。预测时, 只依赖自身及前后词的特征, 不考虑其他词的词性标签结果。
2. word clustering 单词聚类: 将相似的单词分成 k 组以减小特征规模
3. sequence labeling 序列标注:
 - (a) Conditional Random Fields(CRF 条件随机场) 基于大概率出现的标签模式标注序列。CRF 参考词性标签应有的合理排列规律来输出争端序列的标注结果。
 - (b) 找到最大标签序列 $\text{argmax}_y p(y|x)$ 使整段序列一次性完成标注

- Named Entity Recognition(NER 命名实体识别)

识别文本中的实体, 并将其分类到预定义类别中。

识别非结构化的文本中的关键信息来提供上下文和结构。

- Coreference Resolution 共指消解
识别文本中指向同一实体的不同表达。
- Word Sense Disambiguation 词义消歧
词义消歧指根据上下文语境，为文本中的多义词确定其在该场景下的唯一真实含义。
- Semantic Role labeling
语义角色标注/论元识别与标注指为句子中的核心谓语/动词等找出去关联的各类语义角色，并标注每个角色与谓词的语义关系。

7.3 NLP 应用

- Text classification(TC)
文本分类任务自动将文本分类成预定义的标签或主题。
- Machine Translation(MT)
机器翻译任务自动将文本从一门语言转为另一门语言。
方法
 1. Rule-based: 依赖于 linguistic rule(语言规则) 和 grammar(语法)
 2. Statistical: 使用 bilingual text corpora(双语语料库) 进行概率预测。统计性概率翻译让模型从双语语料库中统计规律，统计语料中中文词或短语与英文词或短语的共现概率。统计语料中中文句子结构对应英文句子结构的转换概率。
 3. Neural: 使用深度学习模型
- Information Retrieval(IR)
信息检索是基于用户的询问，在大数据库中发现相关信息的过程。
过程
 1. Indexing 索引：组织数据以快速访问
 2. Query processing 询问处理：解析用户输入以检索相关结果
 3. Ranking：按照查询的相关程度排序结果

方法

1. Boolean Retrieval: 使用布尔运算符进行精确匹配
 2. probabilistic Model: 使用概率对文档进行排序 (BM2.5/TF-IDF)。
不理解文本语义, 仅从词频、文档长度、查询词重要性等表面特征计算查询词出现在相关文档中的概率, 以此作为相关性评分。
 3. 神经网络: 采用深度学习根据相关性对结果进行排序。
- Question Answer
问答系统用于自动回答自然语言提出的问题。
 1. closed-domain(封闭领域): 在一个特定主题领域内回答问题
 2. Open-domain(开放领域): 可以回答广泛主题的问题
 3. Retrieval-based 基于检索 (extractive 抽取式): 查找相关的文档或段落并且抽取答案
 4. Generative Model 生成式模型 (abstractive): 使用神经网络模型直接从知识中生成答案。(神经网络通过大规模的训练参数, 就是学习到的内化知识)
 - Multi-Task Learning(MTL 多任务学习)
 1. 多任务学习训练一个学习同时可以执行多个相关任务
 2. 在任务之间共享知识, 提升各种任务性能。
 - Multi-Modal Learning(MML 多模态学习)
 1. 整合来自多个模态的数据, 通过结合互补信息源提升模型的理解能力。

7.4 NLP 的 evaluation

- IR,TC,NER,POS 等任务
 1. Precision: 所有检索到的 item 中, 正确识别相关 item 所占的比例

2. Recall: 在数据中所有相关的 item 中, 正确识别出相关 item 的比例
 3. F1 Score: 精确率和召回率的 Harmonic mean(调和平均数)
 - Micro F1: 对所有单个实例的精确值和召回率取平均值, 不考虑类别大小, 平等对待所有类别。
 - Macro F1: 分别计算每一个类别的 F1 平均分数并取平均值, 平等对待所有类别。
 4. Accuracy: 正确预测数与总预测数的比例, 适用于平衡的数据集
- MT, TS, GT 等
 1. BLEU(Bilingual Evaluation Understudy 双语评估替补): 衡量生成的翻译和参考翻译中 n 元语法的重叠度。
 - 计算: unigram。BLEU2 要取 1-gram 和 2-gram 的几何平均。
 2. ROUGH(Recall-Oriented Understudy for Gisting Evaluation 面向召回的要点评估替补指标): 评估生成的摘要与参考摘要之间的 n 元语法、单词序列或句子的重叠程度。
 - 如果匹配的单词有 6 个, 在 1-gram 中有 4 个出现在人工参考文本之间, 那么 Rough1 为 $\frac{2}{3}$

8 Word Representation 词汇表征

词表示是将一个单词转换为一个数值向量的形式以便于计算机可以理解, 比较和处理它。

8.1 传统文档表示方法

8.1.1 Bag of Words(BoW) 词带模型的文本表示

- 流程
 1. 将一个文档表示为向量: 每一个文档由长度为 n 的向量组成, n 是语料库中所有不重复的单词的总数。就是说语料库中的 n 个词是有索引的, 我们构建和语料库长度一样的向量来表示一段话, 如果哪一个索引上的词在语料库中出现, 就在相关索引的值上填 1 或者加 1。

2. 向量元素可以捕捉词信息：向量中的每一个元素对应词汇表中的特定词语，并包含该词语在文档中的数值信息。比如，若某个词的索引数字为 5，说明这个词在词汇表中出现了 5 次。
 3. Indicator 指示器 (0/1): 这个向量中展示了词汇是否存在，若索引值为 1 则存在，为 0 则不存在
 4. Term Frequency 词频: 展示单词在文档中出现的次数。
- 缺点
 1. High dimension 高维度: 若词汇表中有上千上万单词，那么表示每一句话就会用上千上万个维度的向量进行表示，但是我们表示的词汇数量又较少，导致向量具有稀疏性且计算成本高。
 2. No context and word order: 忽略了句子中的词序，丢失了语境。
 - 解决方法: Bag of n-gram: 添加词对 (bi-grams) 以捕捉句子中的顺序，但是进一步增加了向量表示维度

8.1.2 Sparse Word Representation 稀疏词表示

- 流程
 1. 语料库可以用一个矩阵进行表示，第一个维度是第几个文档，第二个维度是所有文档的词汇列表。矩阵中的值表示语料库中的词汇 i 在文档 j 中出现频率。这样，每个文档可以用词向量表示；语料库中的每个词也可以用每个文档出现频率向量进行表示。
 2. 可以通过计算文档词向量之间的相似度来对文档进行比较
 3. 可以通过计算词语文档的相似度来比较词语
 4. 相似度使用 cosine similarity 余弦相似度 $\frac{v_1 v_2^T}{\|v_1\| \|v_2\|}$
- 问题
 1. 由于词汇数量多和文档规模大，词汇文档的矩阵维度高且稀疏
- 解决方法 (改进的稀疏词表示方法)
 1. 使用 Singular Value Decomposition(SVD 奇异值分解) 对高维稀疏矩阵进行降维，保留前 k 个奇异值

2. 用 Latent Semantic Indexing(LSI 潜在语义索引) 将文档和单词表示为 k 维度的概念向量。
3. 生成紧凑密集的向量 (word embedding 词嵌入)
4. 使用余弦相似度计算词汇或文档见的相关性

8.2 Word Embeddings: Word2Vec

词表征是将词表示为数字的通用概念。词嵌入式一种特定的词表征形式，使用密集的可学习的向量捕捉词语含义。

- 通过使用语料库中 local co-occurrence within small context windows(局部共现上下文小窗口) 创建词嵌入，防止基于整个文档进行嵌入而丢失局部语义。
- 神经网络方法
 1. 连续词带模型 CBOW: 根据周围词汇预测中心词
 2. Skip-Gram 跳词: 根据中心词预测周围词汇
 3. 具备 Flexible Embedding Size(灵活的嵌入维度): 可以选择嵌入 N 维
 4. Setup 设置: 隐藏层数量，对称上下文窗口，负采样
 5. 输出: 学习的词参数作为词嵌入

8.2.1 Continuous Bag of Words(CBOW) 连续词带模型

聚合上下文词汇的词向量，并且使用组合向量预测中心词。

- 流程
 1. 输入层:

8.2.2 Skip-Gram 跳词模型

使用中心词向量预测其周围每个上下文词。

8.2.3 CBOW vs Skip-Gram

- CBOW 模型训练速度快, 由于其对上下文词汇嵌入组合平均来预测中心词, 因此在大数据集和高频次预测上非常有效。
- Skip gram 单独学习每一个上下文词汇, 因此对于捕捉小数据集和罕见词之间的关系比较有效。

8.2.4 GloVe(Global vector for word representation 用于词表示的全局向量)

GloVe 是一种无监督学习算法, 通过利用语料库中的全局统计信息构建词嵌入。

- Global Co-occurrence 全局共现: 分析语料库以捕捉词共现频率。
- Co-occurrence Matrix 共现矩阵: 构建一个矩阵展示词对在上下文窗口中共同出现的频率
- Optimization 优化: 利用基于共现率的 cost function 学习词嵌入编码语义关系。
- 优势
 1. Semantic capture 语义捕捉: Embedding represent analogies and linear word relationships. 向量表示语义类比和线性词汇关系。就是说, GLoVe 生成的词嵌入能通过线性向量运算还原单词间的语义类比关系。比如国王-男生 + 女生的词嵌入会 = 女王
 2. 利用全局数据进行高效训练。

8.2.5 FastText

FastText 将单词拆分为字符级子词, 以跳词模型和连续词带模型为架构基础, 引入了 subword information. (apple->ap, pp,le)

FastText 将单词表示为 n-gram 字符集, 使其能够捕捉单词的形态特征, 为训练未见过的单词生成词嵌入。因为大多数单词能够拆分为子词的组合嘛。

8.3 Non-contextualized vs contextualized 非上下文相关和上下文相关

- Non-contextualized word embedding 非上下文相关单词表征
 1. 不考虑单词的上下文语境，为每个单词生成单一的静态的向量表征，比如 Word2Vec, GLoVe, 和 FastText。
 2. 这会在不同的语境下导致歧义。
- Contextualized word embedding 上下文相关单词表征
 1. 根据单词的上下文生成单词的不同表示 (dynamic representation)。

8.3.1 ELMo

- ELMo 通过周围的文本生成词的动态嵌入，以捕捉单词在不同语境下的微小差别 (nuances)。
- ELMo 使用一个 2 层的深度双向长段时记忆模型 LSTM 网络，该网络在大型语料库上进行训练，并且以正向和反向上下文进行建模。

8.3.2 BERT

- BERT 模型使用 Transformer 中的 encoder 部分 (多头注意力 + 残差 + 归一化 + 前馈网络 + 残差 + 归一化)。
- BERT 模型通过在大型语料库上进行预训练来学习通用的语言表示。

9 Recurrent Neural Network(RNN 循环神经网络)

循环神经网络通过引入记忆机制对序列进行建模并且捕捉时序依赖。一次处理一个元素，同时维持一个隐藏状态以捕捉来自先前元素的上下文。

9.1 标准 RNN

- 流程：

1. 将句子进行分词并且做词嵌入
2. 对于每一个分词, 从 x_1 到 x_n
3. 在第一个时间步 $t=1$, 输入 x_1 词嵌入, h_0 初始化全 0
 - 输入当前词汇的词嵌入, 并且与前一时间步 (h_0) 的隐藏状态进行拼接。
 - 拼接后输入 RNN 单元, 并且经过 $\tanh$ 激活
 - 得到隐藏状态 h_1
4. 在第二个时间步 $t=2$, 输入 x_2 词嵌入
 - 将 x_2 词嵌入与 h_1 拼接后输入 RNN, 并且通过 $\tanh$ 激活, 得到隐藏状态 h_2
5. 一直循环直到最后一个隐藏状态 h_n 就是核心输出结果

9.2 Long short-term Memory 长短时记忆

标准的 RNN 由于梯度消失, 难以处理长期依赖关系。梯度消失原因: 由于 RNN 每次计算一个元素, 在反向传播的过程中, 随着连续的梯度相乘, 导致梯度逐渐递减为 0, 导致梯度消失。

LSTM 通过引入 specialized gates, 这种门控机制通过使用 sigmoid 函数控制门的输入输出可以选择性地更新记忆:

- Forget gate 遗忘门: Decides what information to discard. 决定什么信息要被丢弃/遗忘
- Input gate 输入门: Decides what new information to store 决定哪些新的信息要被存储
- Output gate 输出门: Determines the next hidden state. 确定下一个隐藏状态。
- Cell State and gates(细胞状态与门控)
 1. 细胞状态: 信息沿着细胞状态传递, 过程中仅发生很小的改变。细胞状态是 LSTM 的主要记忆通道, 信息在传递过程中只通过门控做少量调整, 比如丢弃无用信息, 新增新信息。
 2. LSTM 中的门控: 调控流向细胞状态的信息流。

- Pointwise multiplication 逐点乘法: 过滤信息
- Sigmoid 层 (输出 0 到 1 的数值): 控制通过量。若 sigmoid 值为 0, 说明什么都不通过; 值为 1, 所有东西都通过

3. Forget Gate Layer 遗忘门层

遗忘门决定哪些信息要被丢弃

- (a) 将当前时间步的词嵌入输入, 与前一时刻的隐藏状态输出, 在最后一个维度进行拼接 (就是说输入 5×2 , 五个样本两个特征; 隐藏状态也是 5×2 五个样本两个特征记忆, 在最后维度拼接变成 5×4)。
- (b) 乘以遗忘门权重加上遗忘门偏置得到每个样本的特征的遗忘分数, 维度就是隐藏层维度也就是特征维度 (5×2 , 每个样本每个特征的遗忘分数)。
- (c) 最后使用 sigmoid 激活函数计算遗忘概率, 概率与前一状态的细胞状态相乘计算每个特征的遗忘后的值。
- (d) 公式 $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$

4. Input Gate Layer 输入门层

决定在细胞状态中存入哪些新的信息

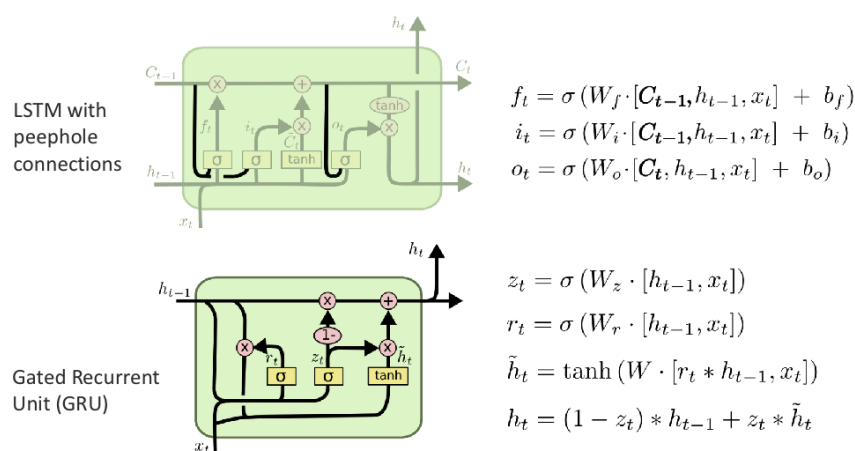
- (a) $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$
 - i_t 决定当前新信息能存多少进细胞状态
 - 拼接前一状态隐藏维度和当前状态词嵌入输入, 乘以新信息权重, 加上偏置, 乘以 sigmoid 得到概率。
- (b) $\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$
 - 候选新信息, 把要加入的新信息值压缩到 $(-1, 1)$ 之间, 避免数值过大导致模型不稳定, 同时让正向语义信息和负语义信息更明确。
- (c) 因此此时细胞状态为: $C_t = f_t C_{t-1} + i_t \tilde{C}_t$

5. Output Gate Layer 输出们层确定细胞状态和隐藏状态的输出

- (a) $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$
 - 输出们通过上一步隐藏状态与当前词嵌入的拼接后计算, 通过学习输出权重, 来得到输出要过滤的概率
- (b) $h_t = o_t \tanh(C_t)$

- 最后的隐藏层等于输出的权重乘当前细胞状态通过 $\tanh$ 和输出权重。最后 h_t 不是 C_t 。 C_t 在输入门更新后就不动了，是长期记忆载体。 h_t 是短时记忆。

9.3 LSTM Variants (LSTM 变体)



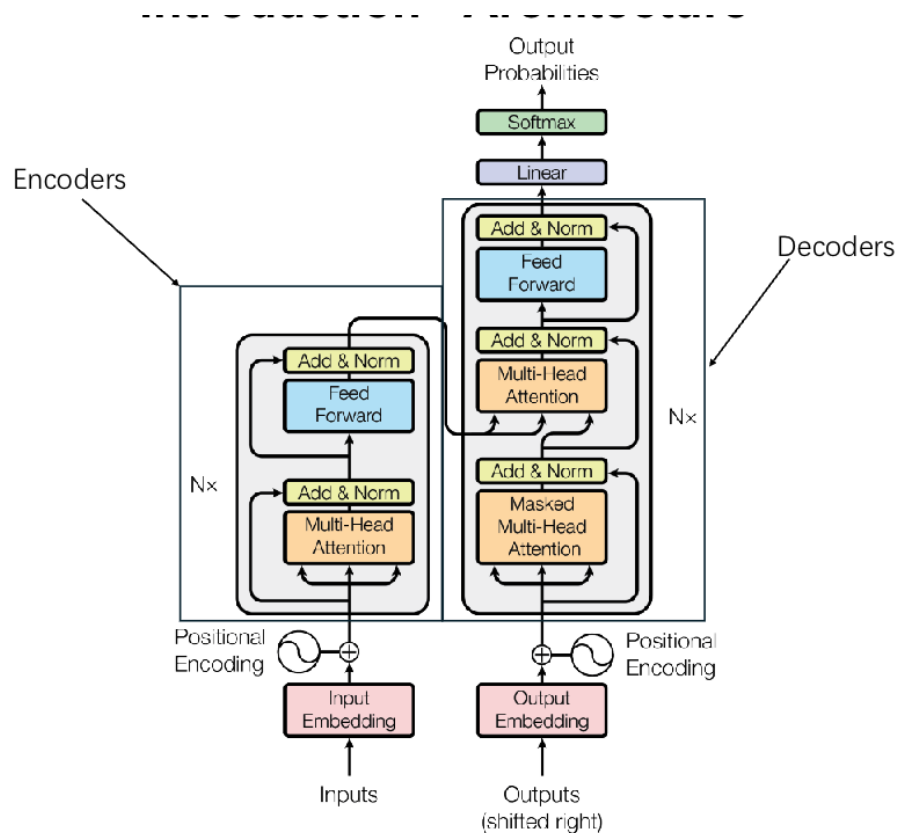
And many more and they exhibit similar performance

图 1: LSTM-variants

9.4 LSTM 优缺点

- 优点: 通过门控机制和跨时间步共享参数，有效处理长期依赖关系
- 缺点: 只能顺序处理，效率低；难以处理极长序列，可扩展性有限。

10 Transformer



The Transformer - model architecture (A Vaswani et al., 2017)

图 2: Transformer

10.1 Transformer 的框架介绍

Transformer 包括 encoder 和 decoder 两个模块

- Encoder 编码器

编码器能够使用多层自注意力和前馈神经网络并行处理输入序列，并生成输入的上下文表征。

- Decoder 解码器

解码器使用 masked self-attention 掩膜自注意力, encoder-decoder attention 编解码自注意力以及前馈神经网络逐步生成输出序列。

- Input/output Embedding 输入输出嵌入
单词或者 token 被转换为密集向量
- positional encoding 位置编码
Transformer 不像 RNN 按照序列顺序一步一步计算隐藏状态。因此其没有固定序列顺序。通过将位置编码加到输入嵌入以保持词顺序。
- Self-attention Mechanism 自注意力机制
自注意力机制能够使序列中的每一个词关注上下文中不同的词 (包括距离较远的词)。
- Multi-Head Attention 多头注意力:
多头注意力接受模型关注句子的不同部分, 提升学习能力。
- Residual Connections and Layer Normalization 残差连接和层归一化
让训练过程更稳定并且加速训练
- Feed-Forward Network 前馈神经网络
注意力之后, 使用全连接层处理数据

10.2 Input/Output Embedding 输入输出嵌入

输入输出嵌入是通过能够处理多种语言和未见过单词的 tokenization(分词) 的方法创建 token 多密集向量表征。

在 Transformer 中输入输出嵌入是 sharing(共享的)。

Tokenization Method

- WordPiece: 使用固定词汇表将生僻词拆分为子词。比如 playing, 词汇表中包含 play 和后缀 ##ing, 则将 playing 拆成这两个部分
- Byte Pair Encoding(BPE 字节对编码): 将频繁出现的字符对进行合并。例子: 有一个词库, 通过计算每个字符和其他字符出现频率, 来对共现字符组成的子词进行排序, 选择共现频率高的子词加入语料库中, 不断迭代直到达到语料库的最大数量或者其他阈值。

- SentencePiece: 对整段文本进行字符序列拆分, 并且通过统计高频字符组合生成子词。例子: unhappiness, 对于词首 un 和 happiness, 拆分为 __un 和 happiness。这样忽略了空格。

10.3 Position embedding 位置编码

标准 transformer 使用正余弦函数表示每个 token 的位置信息, 偶数使用正弦, 奇数使用余弦。最后位置编码将会与 token 嵌入逐元素相加。有助于模型理解词语位置。

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right), PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

- pos 表示序列中第几个 token
- i 表示在嵌入向量中第几个维度
- d_{model} 表示嵌入向量大小, 就是一个 token 用多少维度进行表示。10000 是经验值, 如果太大, 频率减小, 两个维度直接差距变大, 无法捕捉到句子的远程依赖; 如果太小, 频率太快, 两个维度是不是会比较相似?

10.4 Scaled Dot-Product Attention 缩放点积注意力

- 输入词嵌入和位置编码相加之后的结果 (隐藏层的输入)
- 使用 $W^Q \in \mathbb{R}^{d_{model} \times d_Q}, W^K \in \mathbb{R}^{d_{model} \times d_K}, W^V \in \mathbb{R}^{d_{model} \times d_V}$, 三个可学习的权重矩阵与隐藏层嵌入做矩阵乘积, 得到 QKV。Q 是 query 询问, K 是 Key 键, V 是 value 值。
- $Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
 1. 计算 Q 和 K, 用户查询和键之间的相似度
 2. 在第一步初始化 W_Q 和 W_K 让权重的元素方差为 $\frac{1}{d_{model}}$, X 输入是均值为 0 方差为 1, 那么权重乘以 X, 均值为 0, 方差仍然为 1, 因为 X 的维度是 d_{model} , 所以方差求出来是 $d_{model} * \frac{1}{d_{model}}$ 。所以 Q, K, V 均值为 0, 方差均为 1。计算注意力分数时, QK^T 求出来是 token 长度 * token 长度。但是每个元素是 d_k 维度向量的点积, 因此注意力分数的方差为 d_k 。
 3. 因此要除以 $\sqrt{d_k}$

4. 使用 softmax 对注意力相似度进行归一化。
 5. 最后乘以 Value。根据计算出的注意力权重，从原始输入信息中提取并且组合出此 token 新的特征表示。
- 进行残差操作：X+ 新的特征表示
 - 进行归一化操作

10.5 Multi-head Attention 多头注意力

- 将一个缩放点积注意力分成 8 分，求得每一个 token 的 8 个新特征，最后拼接起来乘以权重，得到 8 个新特征融合成一个的特征。最后的特征要与原特征的维度相同，才能做残差操作。
- 多头注意力允许模型并行关注序列的不同部分以捕捉各种类型的关系。
- 多个注意力头独立计算注意力分数后将他们的结果进行拼接和变换
- 此操作能够提升模型学习上下文和依赖关系的能力。
- 为什么缩放点积注意力和残差连接很重要

1. 没有缩放

- 当 d_k 很大时，点积后方差会变大
- Softmax 函数会出现饱和现象，导致梯度消失。比如：一个分数时 100，一个是-100 这样一个导致一些权重很大，一些权重很小，并且 Softmax 在饱和区的梯度几乎为 0.
- 缩放有助于：稳定训练；能够使用更深层更宽的网络

2. 没有残差

- 深度网络会有梯度消失或者梯度爆炸的问题
- 原始输入信息将会在各层中衰减
- 加入残差：保留输入信息；提供一条直接的梯度路径；让模型学习原始特征和加入注意力的特征的精细化内容。

10.6 normalization

10.6.1 Layer Normalization 层归一化

对单个样本的所有特征进行归一化。在自然语言处理中，也就是对单个 token 的词嵌入所有特征进行归一化。

- 适用于序列数据训练
- 在可变的批大小中更加稳定
- Mean: $\mu = \frac{1}{n} \sum_{i=1}^n x_i$
- Variance: $\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$
- Normalized: $\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$

10.6.2 Batch Normalization 批归一化

对批量维度做归一化。就是说对这一批样本中，在同一特征维度下做归一化。如果有 5 个 token，每个 token 有 3 维特征，就对所有 token 的第 1 维特征、所有 token 的第 2 维特征、所有 token 的第 3 维特征分别归一化。

这里 x 表示的是每一个样本，因此每一个样本有多个维度

- 减少批量样本内部的 covariate shift(协变量偏移)，加快训练速度
- 需要较大的 batch size 保证批归一化有效工作
- Mean: $\mu = \frac{1}{m} \sum_{i=1}^m x_i$
- Variance: $\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2$
- Normalized: $\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$

10.7 Position-wise Feed-Forward Networks 逐位置前馈网络

在前馈神经网络中包括一层线性变换 + ReLU 激活函数 + 一层线性变换。 $FFN(x) = \max(0, W_1 x + b_1) W_2 + b_2$

有助于引入非线性，并使模型在处理序列后学习复杂的模式。

10.8 Decoder 解码器

10.8.1 Masked Multi-Head Attention 掩膜多头注意力

掩码多头注意力限制自注意力只能关注当前及之前的 token，确保模型按照从左到右的因果 (causal) 顺序 (autoregressive 自回归) 进行预测。

10.8.2 Encoder-decoder attention

编解码注意力使用前一层的解码器层输出的查询矩阵 Q ，和编码器层输出的键矩阵 K 和值矩阵 V 。这能让解码器在生成序列时，随时回看编码器处理后的输入上下文 (比如翻译前的原文)，确保生成内容贴合输入语义。

10.9 Transformer 的特点

- Parallelisation 并行化

与 RNN 模型相比，Transformer 可以同时处理一整个序列，加速训练和推理

- Long-Range Dependency 长距离依赖

自注意力捕捉一整个序列的关系，获取全局上下文。

- Scalability 可扩展性

Transformer 随着数据和模型规模的大小增加表现好。但是计算复杂性随着序列长度的增加成指数级增长。

11 Transformer-based Language Models

Transfer Learning 迁移学习: 迁移学习是现在数据丰富的任务上预训练模型，再将学到的通用特征迁移到下游特定任务。

11.1 Bidirectional Encoder Representations from Transformers(BERT/transformer 的双向编码器表征)

- 关键特征

1. Pre-trained on large corpora(语料库),(Wikipedia,bookcorpus)

2. 预训练的目标 (半监督训练)
 - Masked language model(MLM 掩码语言模型)
 - (a) 随机掩码输入中的一些 token
 - (b) 模型基于双向上下文学习预测被掩码的 token
 - Next Sentence Prediction(NSP)
 - (a) 预测是否两个句子在文本中是否连续
3. Fine-tuned(微调) for various NLP task(监督训练)
 - (a) 在下游任务上微调模型, 文本分类, 命名实体识别, 问答

11.2 Generative Pre-Training(GPT) 生成式预训练

11.2.1 GPT-1

- GPT1 是生成式的基于 Transformer 解码器架构的模型, 是先进行无监督训练后进行有监督微调的过程。
- 通过序列中的下一个单词 (causal language modeling(CLM 因果语言建模)) 建模自然语言。

11.2.2 GPT-2

- 与 GPT1 架构相似, 但是参数规模增加并且在更大的数据集 (Web-Text) 上训练
- 引入了 probabilistic form for multi-tasking(多任务处理的概率形式)
 1. $P(\text{output}|\text{input}, \text{task})$: 基于输入和任务信息预测输出
 2. 用自然语言文本统一输入和输出, 让模型能以文本交互的方式解决各类问题, 无需为不同任务设计专属架构

11.2.3 GPT3

- 证明了扩大神经网络规模能显著提升模型能力
- In-context Learning(ICL 上下文学习)
 1. 引入了以少样本和无样本形式执行任务的能力。

2. 用自然语言文本教模型做任务，让预训练和实际使用遵循同一个文本交互范式。之前不同的任务需要使用不同的输出头。
3. 给定任务描述和示例 (提示词 `prompt`)，将任务的解决方案预测为文本序列。之前都是生成那种标签而非任务答案。
4. 标志着从预训练大模型转向大语言模型的重要一步。

- GPT 预训练

给定序列 token: $x_1, x_2, \dots, x_T$. 最大化似然 $P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t | x_1, x_2, \dots, x_{t-1})$

- GPT 微调

在 GPT2 之前，输入是文本和位置编码，依次经过掩码多头注意力，残差连接，层归一化，前馈神经网络，残差连接，层归一化，输入多个任务头。

1. Classification 分类任务: `start`(任务开始)+`text`(待分类文本)+`extract`(标识输出结果)，进 transformer 后输入线性层
2. Entailment(蕴含:前提是否蕴含假设):`Start`+`Premise`(前提)+`Delim`(分隔符)+`Hypothesis`(假设)+`Extract`，进 Transformer 输出 linear

为特定任务格式化输入-输出对

11.3 T5

- T5 是基于编码器解码器架构
- 所有的 NLP 任务都可以背理解为 text-to-text 问题
- Span Corruption(跨度毁坏): 在预训练时随机删除文本里的连续片段 (span)，让模型预测这些被删除的内容。删除内容的 token 用 `<mask>` 进行替换。

11.4 其他模型

- DistilBERT
- XLNet

- DeBERTa
- Transformer-XL
- ERNIE
- BART

11.5 Large language model 大语言模型 LLM

Scale: 模型拥有十亿或者数万亿的参数

In-context Learning: 无需微调, 使用少样本和零样本对任务进行适配。
(这里的意思是我们之前在做任务是要先进行下游任务的微调, 但是 LLM 的预训练足以让大语言模型拥有较好的零样本和少样本能力)

Emergent ability 涌现能力: 展现出小模型所不具备的推理能力, 创造力和域适应能力

- Pre-training
从大量标注的数据中学习一般语言模式
- Post-training(Fine tuning)
使模型对其人类的意图, 使模型适应对应任务

1. 现有大语言模型

- LLAMA
- Claude
- ChatGPT: 对话数据集上进行微调, 使模型具备多轮交互的能力;
使用 reinforcement learning from human feedback(RLHF)
- Gemini
- Grok
- Deepseek
- Qwen
- Doubao

2. Scaling LLMs 的挑战

- Model size and computational cost: 大型模型需要更高性能的 GPU, 导致计算成本昂贵。
- Energy consumption: 训练大模型消耗大量电力
- Latency and real-time usage: 大模型往往难以获得实时的结果, 对于许多在边缘设备的应用至关重要

3. 高效人工智能优化策略

- (a) Quantization 量化: 降低模型参数的数值精度来节省内存占用, 加快计算速度。
- (b) Knowledge Distillation 知识蒸馏: 训练更小的学生模型来模仿更大的教师模型的行为, 在降低模型复杂度的情况下实现相近的性能。
- (c) Model Compression 模型压缩: 在保持模型有效的同时, 通过剪枝 (pruning) 或权重共享 (weight-sharing) 减小模型规模
- (d) Efficient Architectures 高效架构: Mixture of Expert(MoE 专家混合模型), sparsity.

4. Mixture of Expert(MoE)

将前馈网络层拆分为多个并行的专家, 在保证模型计算不变的情况下扩展模型能力

- (a) 专家混合模型是一种稀疏的大语言模型架构, 包含许多专家子网络。
- (b) MoE 架构中, 路由模块会为每个输入 token 选择 top-k 个最匹配的专家, 因此每个 token 仅激活一小部分专家。
- (c) 每个专家有不同的专长
- (d) 专家混合模型在不增加计算量的前提下扩展模型能力

11.6 未来发展方向

- Inference-time reasoning(CoT, planning, reflection)
- Tool and retrieval-augmented models(RAG, MCP)
- Agentic AI: models that act, verify and adapt

12 Reasoning and Agentic LLMS

模型的多步推理和任务分解能力由思维链等其他技术实现。

12.1 Chain of thought (CoT 思维链)

- 思维链是一种提示技术，引导模型在生成最终答案之前对问题一步一步思考。它帮助模型在不借助外部工具的情况下进行模型内步推理。
- Variants of CoT
 1. Zero-shot CoT(零样本思维链): 无示例提示
 2. Few-shot CoT(少样本思维链): 提供少量带完整推理步骤的示例
 3. Self-consistency(自一致性): 对多个思维链进行采样并且汇总结果，通过投票输出最优解
 4. Hidden CoT(隐藏思维链): 内部推理，输出最终答案

12.2 Tree of Thought(ToT 思维树)

Reasoning= 在思维空间中进行搜索

- 思维树会让模型探索多个候选思路，对其进行评估，并选择或扩展最佳思路。
- 流程:
 1. Thought Generation 思维生成: 生成下一个步骤的多个候选思维
 2. Thought Evaluation 思维评估: 对 partial solution(部分解) 进行打分，一般使用 self-evaluation 自评估 (模型自己推断候选解法是否可行，是否产生矛盾等), heuristic 启发式评估 (用简单规则快速打分。比如条件是否满足约束，或者数学公式是否正确), verifier 外部验证器 (调用专门的工具进行验证。比如使用计算器验证计算中间过程。)

12.3 ReAct=Reason+Act

ReAct 是大模型推理和行动的主流框架之一。指导智能体如何分步调用工具。这个框架规定智能体可以通过 Thought->Action->Observation 进行循环迭代。

比如说，我现在要知道明天要买什么衣服，模型会思考我要做什么，然后行动比如打开天气软件，查看天气之类的；再思考一步打开淘宝，它帮你搜索淘宝找衣服，然后给你链接。而不是简单地进行推理。

12.4 Agentic LLM 具备代理能力的大语言模型

- Agent(智能体) 是一个利用人工智能模型与环境交互以实现用户定义目标的系统。通过结合 reason, planning 和 the execution of action 来完成任务。
- Agentic LLMs(智能体化大语言模型) 是一个使用大语言模型作为核心推理引擎的系统
 1. 能够理解自然语言：以有意义的方式解读并相应人类指令
 2. reason and plan: 分析信息，做决定并且制定解决问题的策略
 3. 与环境 interact：收集信息，采取行动，观察行动结果
- Thought action Observation:think plan act observe

12.5 Model Context Protocol(MCP 模型上下文协议)

MCP 是一个将人工智能应用程序连接到外部工具的 standardized 协议。

- MCP 主要是客户端-服务器模式
- MCP client 向服务器发起请求，负责调用工具，查询资源，插入提示词
- MCP server 提供服务，暴露工具、资源和提示词模版
- 支持本地连接和远程部署

- Tool(模型控制): 模型可直接调用的函数, 用于执行操作。检索, 更新数据库
- Resource(应用控制): 提供应用的数据。文件, 数据库
- prompts(用户定义): 预定义的交互模版, 规范 AI 输出与交互格式。文档问答
- 安全问题:
 1. 智能体不仅生成内容, 而且采取实际行动, 可能引发实际危害
 2. 智能体优化目标与实际用户需求不一致, 导致错误或无效结果
 3. 智能体滥用工具或越权使用工具
 4. 智能体基于错误假设 (幻觉信息) 采取行动
- 解决方案:
 1. Human-in-the-loop approval for critical action: 重要操作需人类确认后再操作, 避免智能体自主触发高风险行为
 2. Tool sandbox and least privilege: 限制智能体调用工具的权限范围, 把工具操作隔离在安全环境中, 防止滥用
 3. Explicit stop time and cost
 4. External verification
 5. Logging, traceability, and replay

13 Reinforcement learning 强化学习

13.1 强化学习框架

- 强化学习是一种解决控制问题的框架, 它通过构建智能体来实现, 这些智能体通过 trails and errors(试错) 与环境进行交互并且 receiving rewards 作为唯一反馈来从环境中学习。
 1. Agent interaction 智能体交互: 人工智能体通过与环境交互进行学习
 2. Trail and error 试错: 通过行动和输出结果实现学习

3. Reward feedback 奖励反馈: 接收正面和负面的奖励作为行为的反馈
4. Natural Inspiration 自然启发: 模仿人类和动物通过交互的方式进行学习
5. Goal 目标: 学习如何采取行动以最大化累计奖励, 一条动作链的累积奖励被称为期望奖励 (expected return)。每一步的即时奖励被称为 reward。

- 强化学习主要变量: state, action, reward, next state

13.2 强化学习的概念

- Observation and state(观测与状态)

Observation 和 state 是智能体从环境中获取的信息。

1. State: 状态是对环境的完整描述, 包含所有后续发展的关键信息, 不存在任何隐藏信息。
2. Observation: 由于环境中含有隐藏信息, 观测是智能体从环境中获取的部分信息。比如环境中含有敌人的下一步动作这种随机信息, 难以预测。

- Action space 动作空间

动作空间是环境中所有可能动作的集合。

1. discrete action space 离散动作空间: 动作是有限的
2. Continuous action space 连续动作空间: 可能的动作数量是无限的

- Types of RL tasks

1. Episodic task: 有明确开始和结束状态的任务。
2. Continuous task: 没有终止状态且无限期运行直到人手动停止的任务。

- exploration 探索 vs exploitation 利用

1. exploration: 尝试不同的动作发现潜在的更高的奖励
 2. exploitation: 利用已知的信息最大化即时奖励
- Markov Decision Process(马尔可夫决策过程)
 1. 智能体仅仅通过当前的状态来决定要执行什么动作，不需要历史的状态和动作。
 2. Policy 策略 π 是从状态 S 映射到动作 A 的函数，这函数指定了在什么状态下采取什么动作。
 3. 目标是最大化 accumulative discounted reward(累计折扣奖励) 发现最优策略 π^*
 4. $R(\tau) = \sum_{t \geq 0} \gamma^t r_t$ 。 τ 是状态动作序列； γ 是折扣率，通常为 0.95 到 0.99 之间，如果折扣率设置很小那么奖励主要由前几个动作和状态决定，因为随着步数 t 的增大，之后步数的奖励会逐渐减小； r_t 即时奖励
 - reward function 奖励方程一些问题
 1. 将奖励与目标对齐：确保奖励准确反映期望结果
 2. 避免 Award hacking(奖励欺骗)：防止智能体利用 loopholes(漏洞) 以非预期的方式最大化奖励
 3. 平衡 immediate reward and long-term rewards(即时奖励和长期奖励)：确保智能体重视长期成功而非较高的即时奖励
 4. 处理复杂和多任务目标：设计能够涵盖任务所有方面且无冲突的奖励机制
 5. 扩展性与自适应性：创建函数与环境变得更复杂时仍能运行良好的奖励函数
 - Delayed reward(延迟奖励)

当智能体没有通过它的行为获得即时奖励，且其结果只有在一系列后续行为和时间步之后才会 released，就会出现延迟奖励。

 1. 智能体需要学会将其行为与未来关系联系起来，这通常需要跨越较长的时间范围，使得学习变得复杂且不够直接

2. 通过 exploration(随机选择动作执行) 和 exploitation(选择已知能够产生奖励的动作)
- 寻找最优策略的方法
 1. Policy-based methods
Direct learning 直接学习: 教智能体直接通过当前状态选择动作
 2. Value-based methods
Indirect learning 间接学习: 训练智能体通过评估状态的价值并且选择能够导向更高价值状态的行为。
 - Value/return vs Reward
 1. 状态或者状态动作对的价值表示 expected accumulated reward。
是智能体从该状态或者状态动作对开始, 遵循自身策略不断行动获得的预期累计奖励。
 2. 奖励是智能体在特定状态下执行动作后从环境中获得的即时奖励。
 - Policy-based method 基于策略的方法
策略函数是直接学习得到的, 定义了每一个状态到最佳动作的映射
 1. Deterministic Policy 确定性策略
对于给定的状态, 都有唯一确定执行的动作
 2. Stochastic Policy 随机策略
为每个状态提供动作概率分布。比如说一个状态, 执行动作 1 的概率是 30%, 执行动作 2 的概率是 70%。
$$\pi(a|s) = P(A|s)$$
 - Value-based method 基于价值的方法
学习一个策略函数, 该函数将一个状态映射为处于该状态的期望价值。
状态的价值指智能体从该状态出发并遵循策略所能获得的 expected discounted return(期望折扣累计回报)。通过找到策略, 以发现价值最高的状态。

- Off-policy 离线策略 vs Online-policy 在线策略
 1. Off-Policy: The acting policy 和 updating policy 不同。(Q-Learning)。
就是说, Q 学习中, 智能体做动作是根据 ϵ 贪心规则, 不断探索和利用; 而它更新是根据动作策略收集到的经验, 选择贪心算法得到最优策略。
 2. On-Policy: 动作策略和更新策略相同。

13.3 Deep Reinforcement Learning 深度强化学习

深度强化学习引入深度神经网络来解决强化学习问题

- 两种基于价值算法
 1. Q-Learning(传统强化学习)
 - 使用 Q 表存储每个状态的动作价值
 - 基于表中的 Q 值进行每个状态的动作选择
 2. Deep Q-Learning(DRL)
 - 使用神经网络替换 Q 表来近似 Q 值
 - 神经网络在大的连续的状态空间中具备更好的泛化性

13.4 Q-learning

- Q 学习是一种离线策略基于价值的方法, 它使用时间差分 (temporal difference) 方法训练动作-价值函数。
- Q 函数是状态-动作价值函数, 决定了处于特定状态下采取特定动作的价值, 这个价值也被称为 Q-value。
- 目标: 找到最优的 Q 函数, 来得到最优策略
- $\pi^*(s) = \operatorname{argmax} Q^*(s, a)$
- 使用 Q 表, 记录当前状态做所有动作的 Q 值 (state-action pair values)
- 算法流程
 1. 初始化 Q 表为 0, 行数为状态个数, 列数为动作个数

2. 适用 ϵ -greedy policy 选择一个动作。以 ϵ 的概率执行 exploration(随机选择动作), 以 $1-\epsilon$ 概率执行 exploitation(选择一个贪心动作)。
3. 训练初期 ϵ 较大, 模型探索概率高, 目的是多尝试、收集环境信息; 训练后期 ϵ 减小, 模型倾向于选择最优策略。
4. 更新状态价值函数: $V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$. $V(S_t)$ 是当前状态的价值, α 是学习率, R_{t+1} 是当前状态执行动作之后的即时奖励, $\gamma V(S_{t+1})$ 是下一状态的折扣价值 (约等于累计期望价值)
5. 更新动作价值函数: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$. $\max_a Q(S_{t+1}, a)$ 表示在下一状态 S_{t+1} 的所有可能动作 a 中选择能让 Q 值 (长期收益期望) 最大的动作,

- 缺点

1. 可扩展性差: 在小型的状态空间中表现良好, 但在复杂环境中变小不佳

- 优化

1. 用可学习的参数化的 Q 函数替换 Q 表
2. 使用神经网络来近似给定状态下每个动作的 Q 值, 也被称为 DQL。

13.5 Deep Q-learning

深度 Q 学习使用神经网络对每个状态下每个动作 Q 值的近似计算。

- 在视觉任务下的 DQL 中, 通过对图像帧的堆叠提供时序信息。并且对图片进行降维和灰度化来减小计算量。
- 训练流程:
 1. 采样: 执行动作并且将经验元组存储在 replay memory 中, 经验元组有 (状态, 动作, 奖励, 下一状态)
 2. 训练: 随机采样一批元组, 并使用梯度下降法进行学习。
 3. Q-target: $y_j = r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-)$ 计算即时奖励和加权估计最优 Q 值。

4. Q-loss: $Loss = y_j - Q(\phi_j, a_j; \theta)$, 通过最小化当前 Q 网络预测值与目标值 y_j 之间的 (均方) 误差, 使 Q 网络逐渐满足贝尔曼最优方程。

- 深度 Q 学习不稳定性:

1. 由于结合基于神经网络的非线形 Q 值函数
2. 使用估计值更新目标值 (bootstrapping)

- 解决方法

1. Experience replay(经验回放)
 - 在回放缓存中存储和回放经验
 - 从相同的经验中多次学习
 - 避免 catastrophic forgetting
 - 打破连续经验之间的相关性
2. Fixed Q-Target(固定 Q 目标函数)
 - 使用具有固定权重的单独目标网络来稳定 Q 值更新
 - 每 N 步更新一次目标网络以减少震荡
3. Double deep Q-Learning(双深度 Q 学习)
 - 使用两个神经网络避免 DQL 中 Q 值被过高估计的问题
 - 使用 DQN 网络选择最佳动作
 - 使用目标网络计算该动作的 Q 值